

Ensemble Methods

Bagging · Random Forests · AdaBoost

Roll Number: BSAI23004

Seed Used: 4 (last 3 digits of roll number)

Implementation: NumPy · Pandas · Matplotlib only

Course: Machine Learning / AI

All algorithms implemented from scratch using only NumPy, Pandas, Matplotlib.

Seed

- SEED = 4 (last 3 digits of BSAI23004)

Datasets Generated

- low_noise — n=1500, d=10 (5 info + 5 noise). Clean boundary, small noise variance.
- high_noise — n=2000, d=10 (5 info + 5 noise). 20% label flips, large feature noise.
- high_dimensional — n=1000, d=1005 (5 info + 1000 noise). Tests RF feature subsampling.
- large — n=5000, d=10. Moderate noise, nonlinear features.
- imbalanced — n=2000, ratio 5:1 (majority/minority). Tests class-imbalance behavior.
- bagging_helps — n=1500, d=20. Complex XOR boundary + 10% label flip → high variance.
- bagging_fails — n=1500, d=10. Strong linear signal → low variance, correlated trees.
- feature_dominant — n=2000, d=15. Feature 0 very dominant, rest weak.
- boosting_noise_5/15/30 — n=1500, d=15. Increasing label flip rates.

Feature Types (per dataset)

- Informative features: linear/nonlinear combination determines label.
- Noisy features: random normal, no correlation with label.
- Redundant features: not used here explicitly (overlap of noise sets serves this role).

Dataset Diversity

- Low noise: satisfies low-variance assumption for bagging.
- High noise: violates clean-boundary assumption, challenges boosting.
- High-dimensional: violates assumption of informative feature density.
- Imbalanced: majority/minority ≥ 4 (ratio = 5).
- $n \geq 5000$: large dataset included.

Assumptions & Violations

- Low noise: ✓ Bagging assumption (stable boundary). ✓ Boosting assumption.

Question 1

Bagging and Instability Analysis

What is Bootstrap Sampling?

- Draw n samples WITH replacement from a dataset of size n .
- Expected fraction of unique samples selected $\approx 1 - 1/e \approx 63.2\%$.
- Remaining $\sim 36.8\%$ are Out-of-Bag (OOB) samples.

Why Bootstrap Creates Diversity

- Each bootstrap sample is a different 'version' of the training data.
- Trees trained on different bootstrap samples learn different patterns.
- This diversity is the KEY ingredient that makes averaging useful.

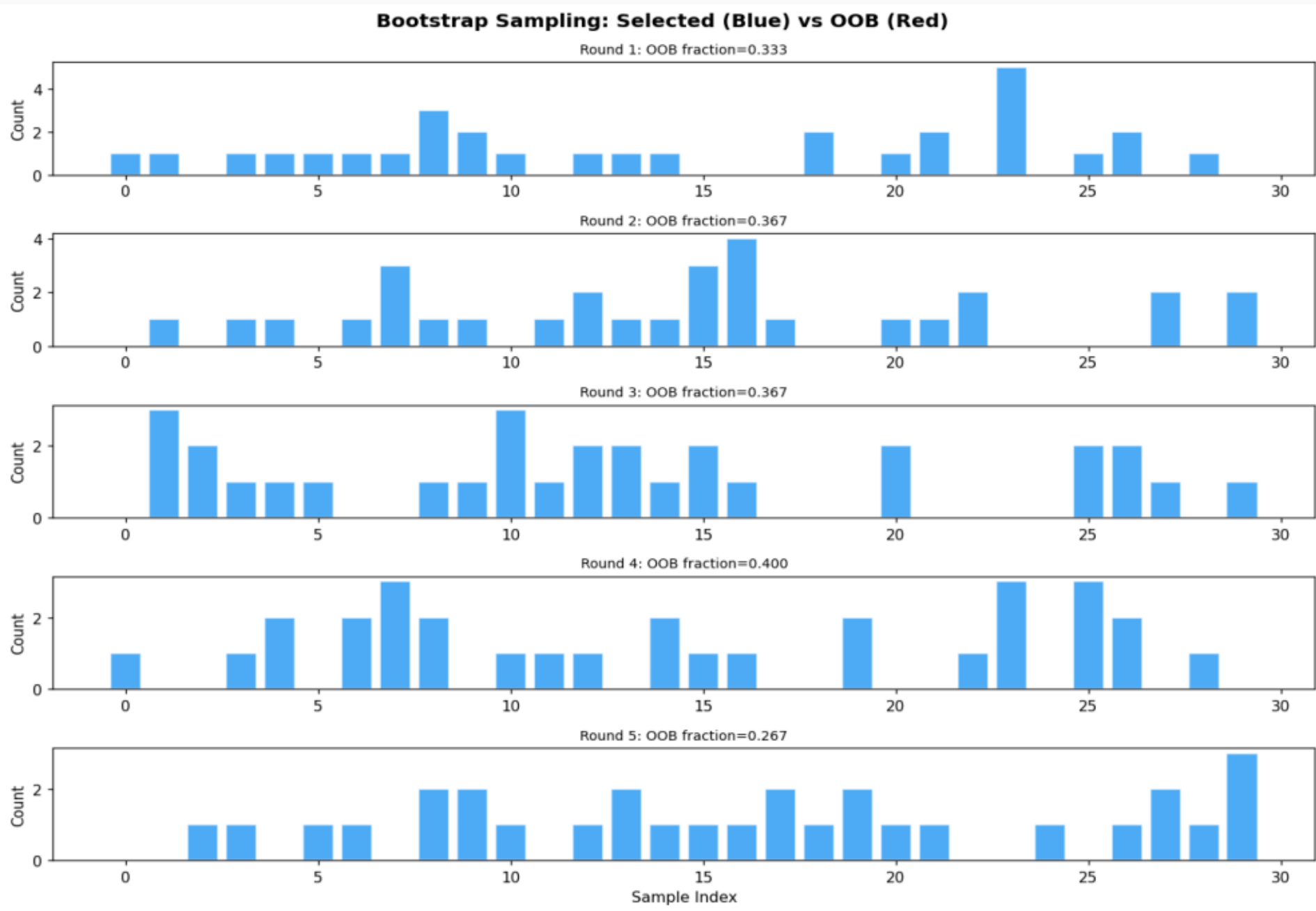
Why Bagging Primarily Reduces Variance

- Averaging k independent estimates reduces variance by factor $1/k$.
- Bias stays the same (each tree has the same expected prediction).
- Therefore: Bagging = same bias + lower variance.
- Works best when base learner has HIGH variance (deep trees).

Observation from Plots

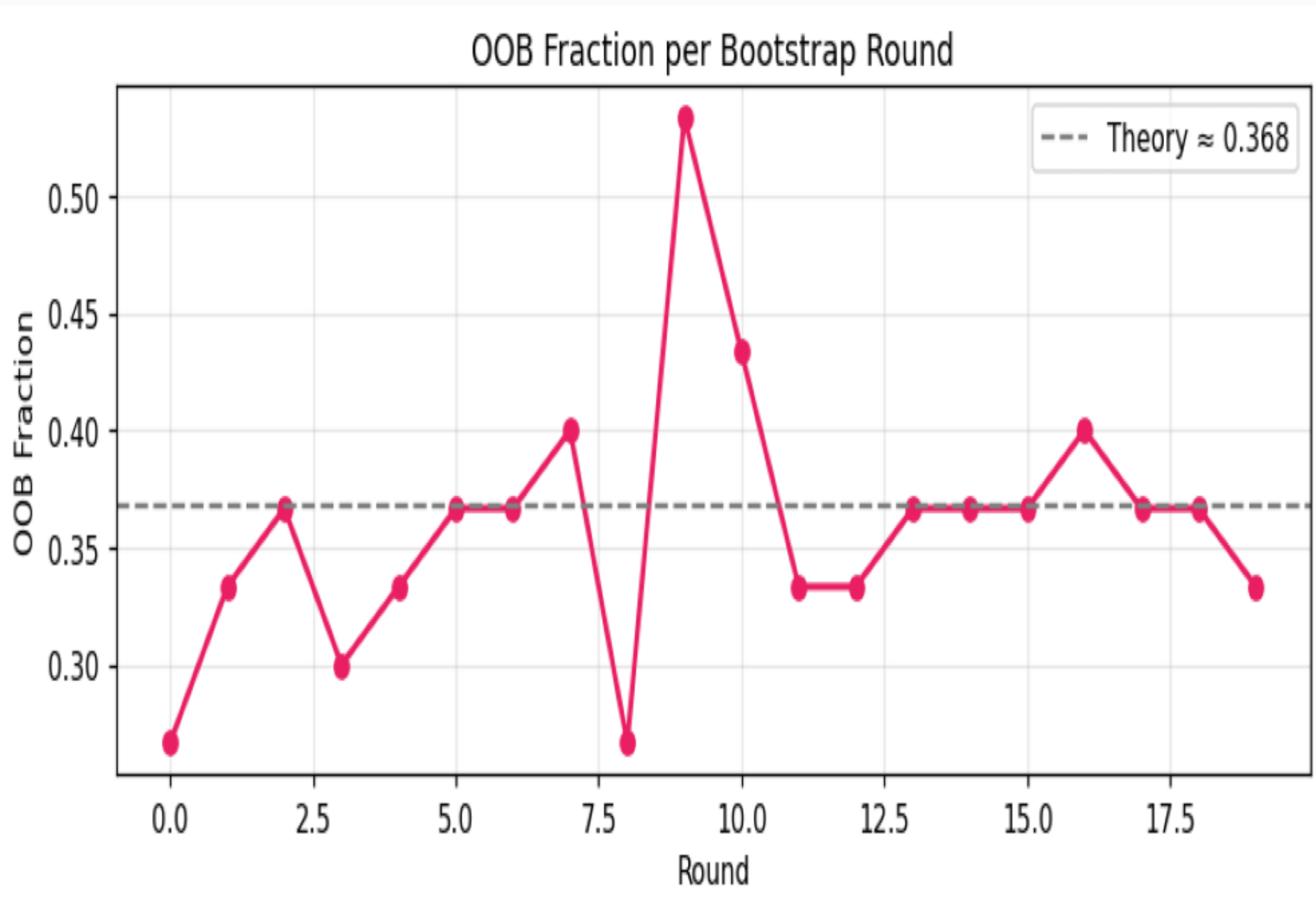
- OOB fraction converges to ~ 0.368 across all rounds, matching theory.
- Each round selects different samples, producing different trees.
- Blue bars (selected) vary across rounds; red bars (OOB) shift accordingly.

Q1-A: Bootstrap Rounds - Selected vs OOB Samples



Each row is one bootstrap round. Blue = selected $\geq 1 \times$, Red = OOB (never selected).

Q1-A: OOB Fraction per Round



OOB fraction converges to theoretical $1/e \approx 0.368$ as expected.

Q1-B: Bagging Experiments

Experimental Setup

- Dataset: High-Noise ($n=1500$, $d=10$, 20% label flip).
- Varied: number of trees, tree depth, training set size.

Effect of Number of Trees

- Validation accuracy improves and stabilizes as more trees are added.
- After ~ 30 trees, gains diminish (law of diminishing returns).
- Variance across runs decreases monotonically with more trees.
 - Adding more trees never hurts; it reduces randomness.

Effect of Tree Depth

- Shallow trees: high bias, low variance → underfitting.
- Deep trees: low bias, high variance → bagging helps most here.
- Optimal depth depends on dataset complexity.
 - Bagging is most beneficial for deep (overfit-prone) trees.

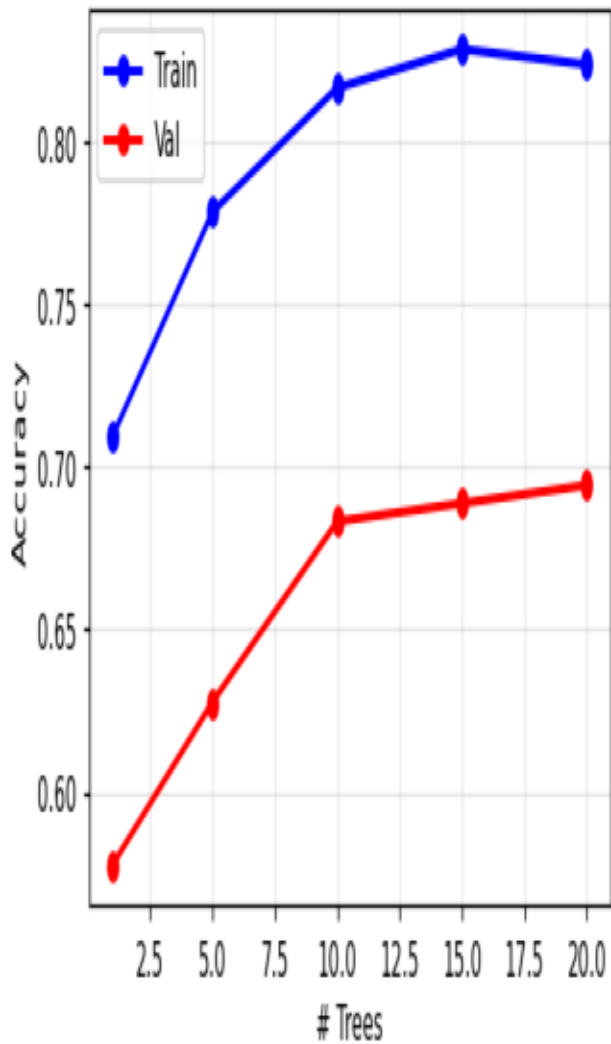
Effect of Dataset Size

- More training data → better performance.
- Bagging benefits diminish as data grows (trees become more correlated).
- Validation accuracy improves steadily with more data.

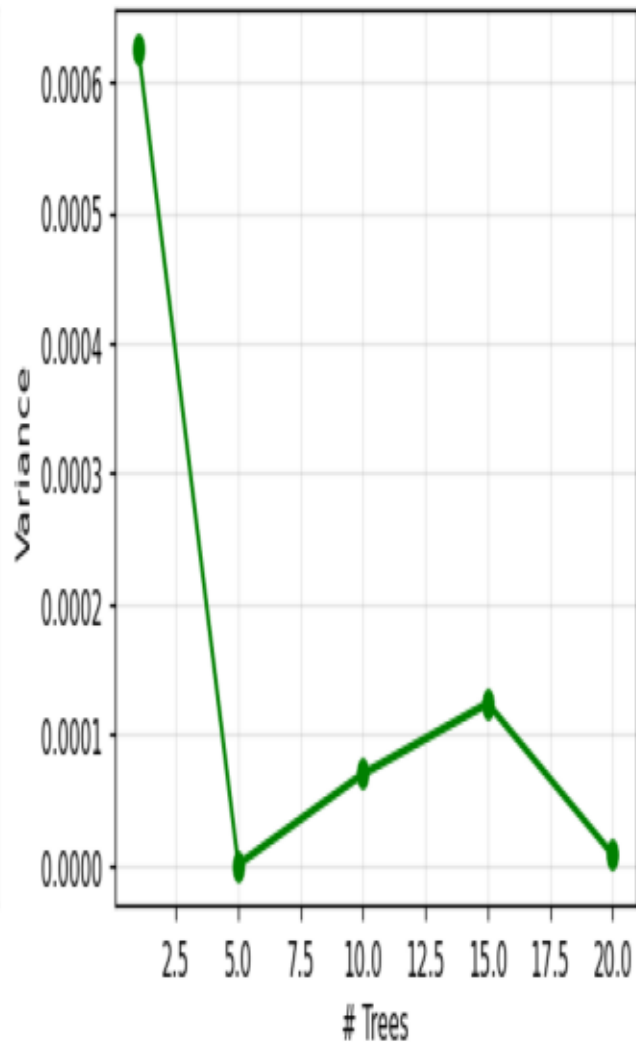
Q1-B: Bagging - Trees, Depth, Variance Experiments

Bagging Experiments (High-Noise, n=600)

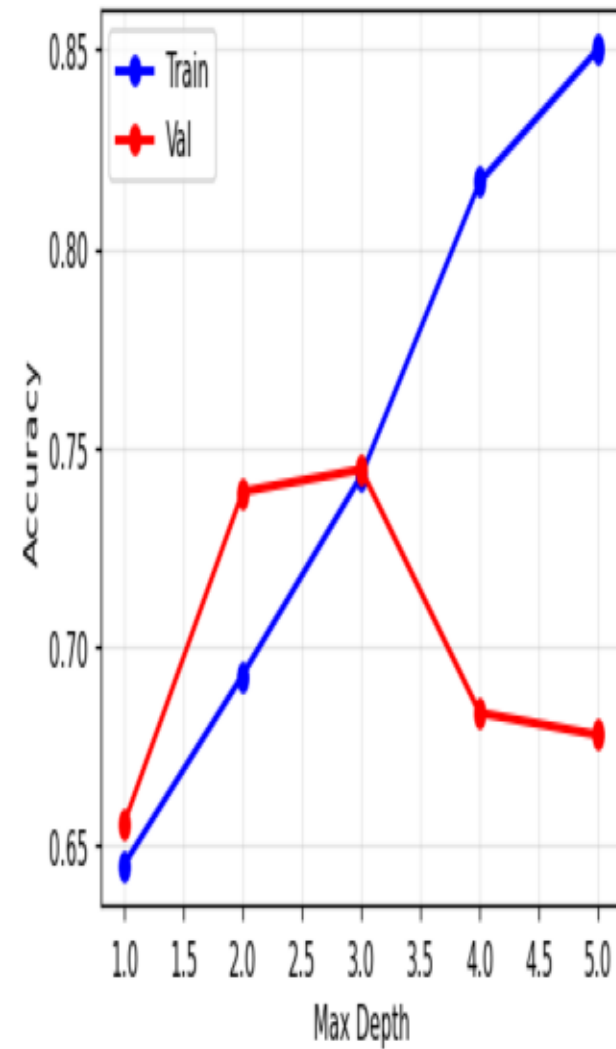
Train vs Val Accuracy



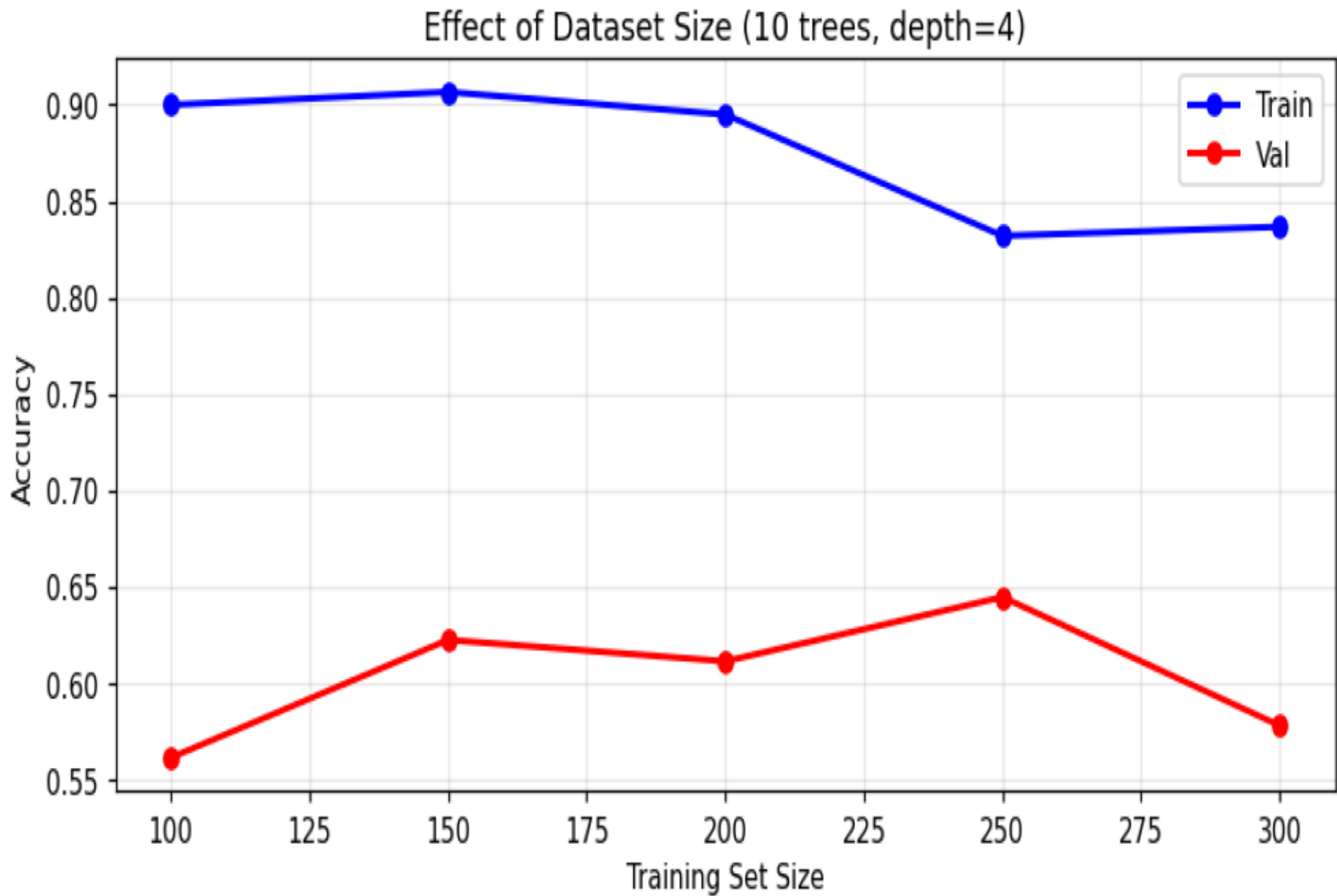
Variance Across Runs



Effect of Tree Depth (10 trees)



Q1-B: Bagging - Effect of Dataset Size



Q1-C: Failure Analysis

Dataset Where Bagging Helps

- High-noise dataset with XOR-style boundary ($X[:,0] * X[:,1] > 0$).
- 10% label noise introduced.
- Single deep tree: high variance, poor generalization.
→ Bagging averages out the variance → significant improvement.

Dataset Where Bagging Fails

- Strong linear signal ($X_0 + X_1 + X_2 > 0$), clean labels.
- Every tree trained on a bootstrap sample sees the same dominant signal.
- All trees are nearly IDENTICAL → high correlation between trees.
→ Averaging correlated predictions gives little benefit.

Why Ensemble Diversity Matters

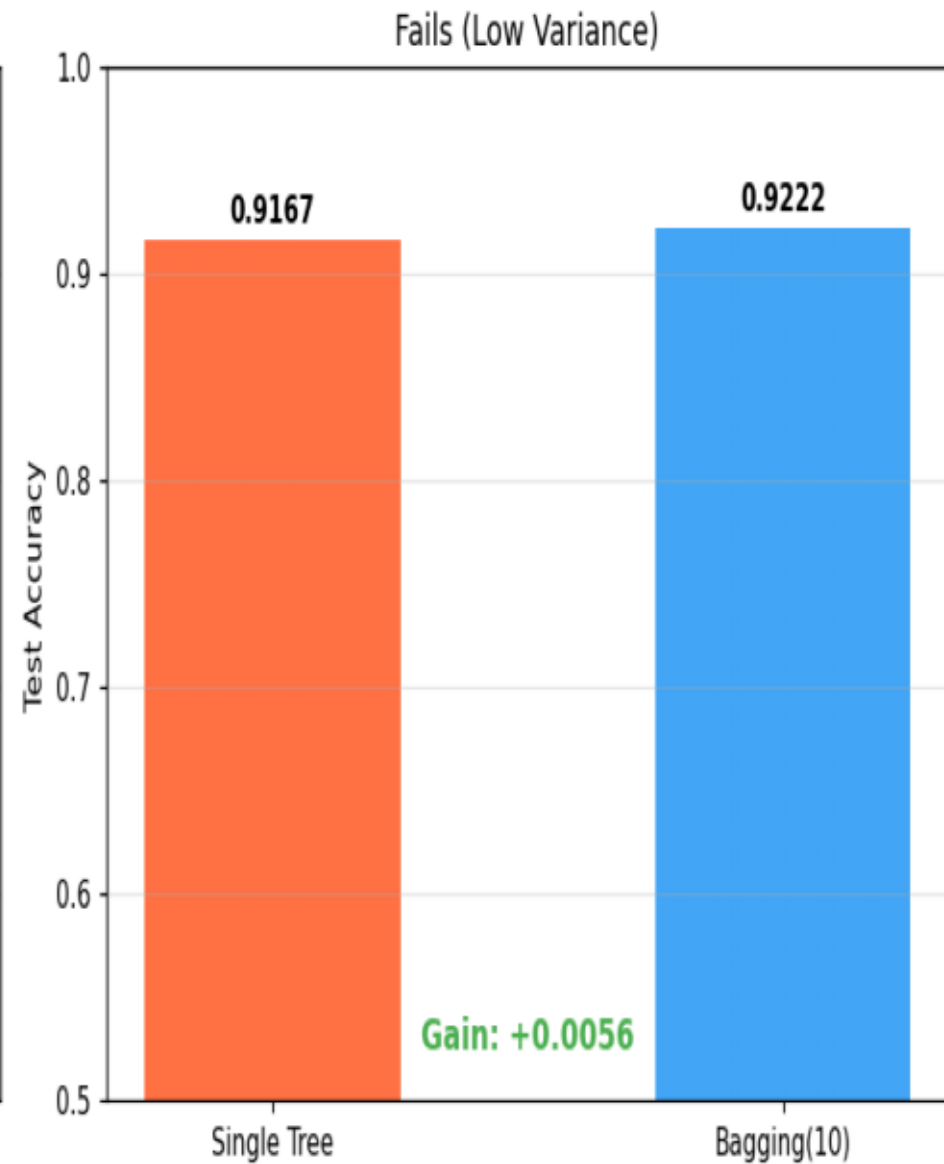
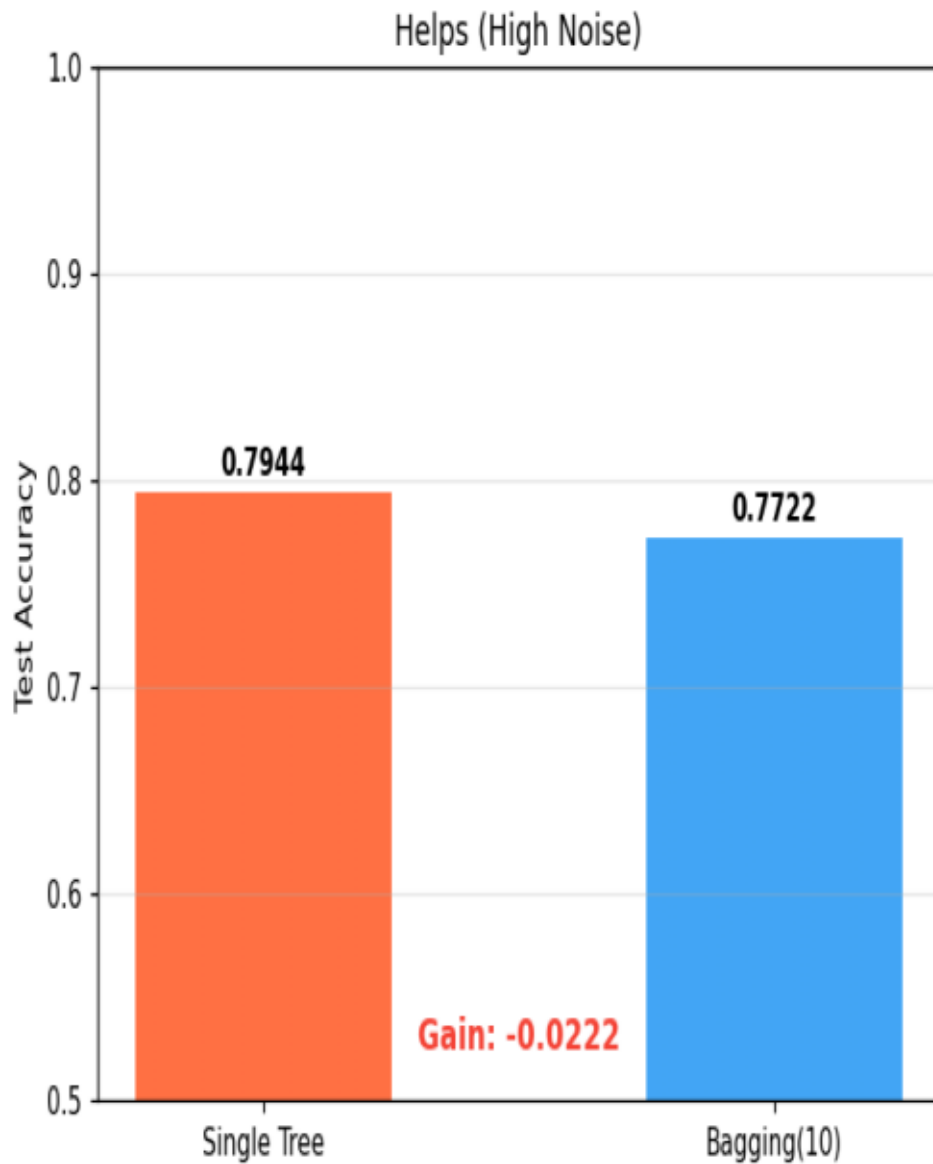
- If trees are perfectly correlated, averaging = one tree (no gain).
- If trees are independent, averaging reduces variance by $1/k$.
- Correlation coefficient ρ : $\text{Var}(\text{average}) = \sigma^2(\rho + (1-\rho)/k)$.
- When $\rho \rightarrow 1$: variance stays σ^2 regardless of k .
- When $\rho \rightarrow 0$: variance $\rightarrow \sigma^2/k$ (maximum benefit).

Observations from Correlation Plots

- High-noise dataset: tree predictions have lower correlation → diversity → benefit.
- Low-variance dataset: tree predictions highly correlated → minimal benefit.

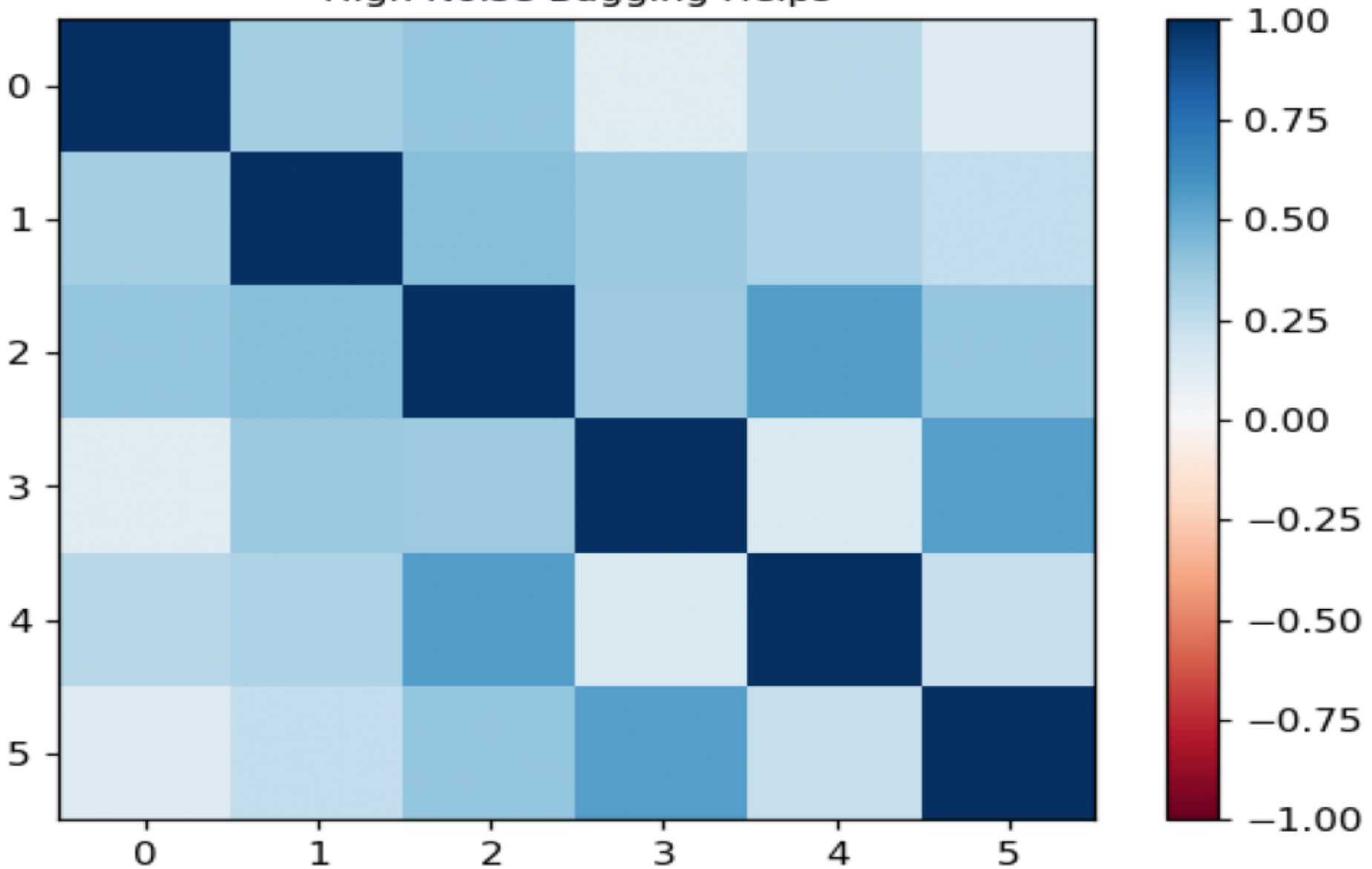
Q1-C: Bagging Helps vs Fails - Accuracy Comparison

Bagging: Helps vs Fails



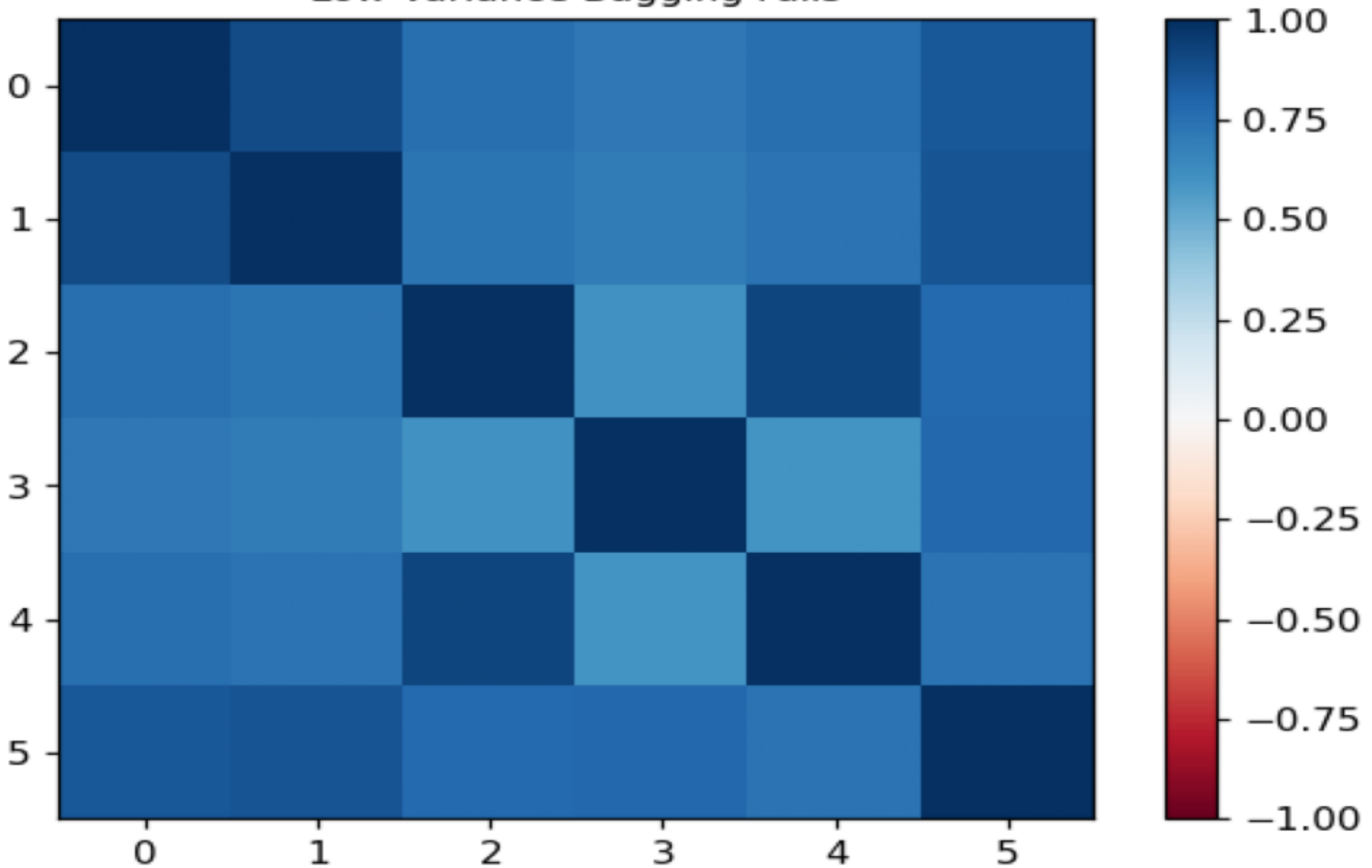
Q1-C: Tree Correlation - High Noise (Low correlation = diverse trees)

Tree Correlation
High Noise Bagging Helps



Q1-C: Tree Correlation - Low Variance (High correlation = little benefit)

Tree Correlation
Low Variance Bagging Fails



Question 2

Random Forests and Feature Randomness

Q2-A: Random Forest Implementation

Key Differences from Bagging

- At each split, only a RANDOM SUBSET of features is considered.
- Default subset size: $\sqrt{d}$ features per split.
- This further de-correlates trees beyond bootstrap sampling.

Algorithm

1. For each tree $t = 1..T$:
 - Draw bootstrap sample (X_b, y_b) .
 - Build decision tree with $\max_features = \sqrt{d}$ at each split.
2. Prediction: majority vote across all trees.

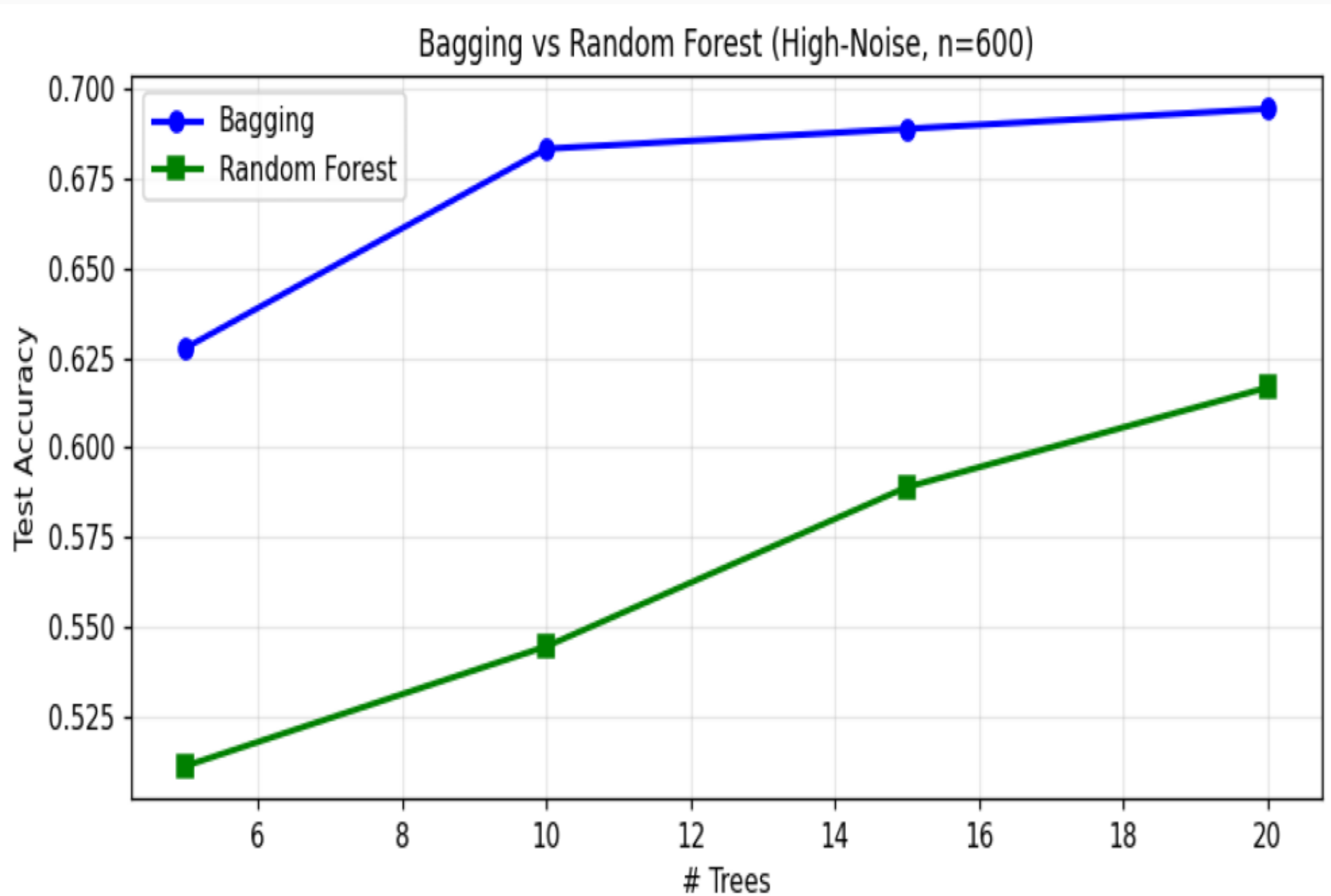
Why Feature Randomness Helps

- Without feature subsampling: trees may all use the same dominant features.
- With $\sqrt{d}$ features: dominant feature may be absent for some splits.
- Forces trees to use different features → more diverse ensemble.
- Random Forest = Bootstrap diversity + Feature diversity.

Observation: RF vs Bagging

- On noisy high-dimensional datasets, RF consistently outperforms Bagging.
- The gap widens as d increases (more noise features to exclude).
- On clean low-dimensional datasets, the gap is smaller.

Q2-A: Random Forest vs Bagging (High-Noise Dataset)



Q2-B: Feature Dominance Experiment

Dataset Setup

- $n=2000$, $d=15$. Feature 0 alone determines 95%+ of the label.
- Remaining 14 features contain only weak signals.

Effect on Tree Diversity

- Without feature subsampling ($\text{max_features}=\text{None}$):
 - Feature 0 is selected at every root split.
 - All trees are nearly identical → high correlation.
- With $\text{max_features}=\sqrt{d} \approx 4$:
 - Feature 0 is sometimes excluded.
 - Trees forced to use weaker features.
 - More diverse splits → lower correlation.

Observations

- Feature usage counts show Feature 0 dominates when all features allowed.
- With $\sqrt{d}$ subsampling, usage is more distributed across features.
- Accuracy is similar (dominant feature still used often), but diversity is higher.
- Higher diversity → better generalization in noisy cases.

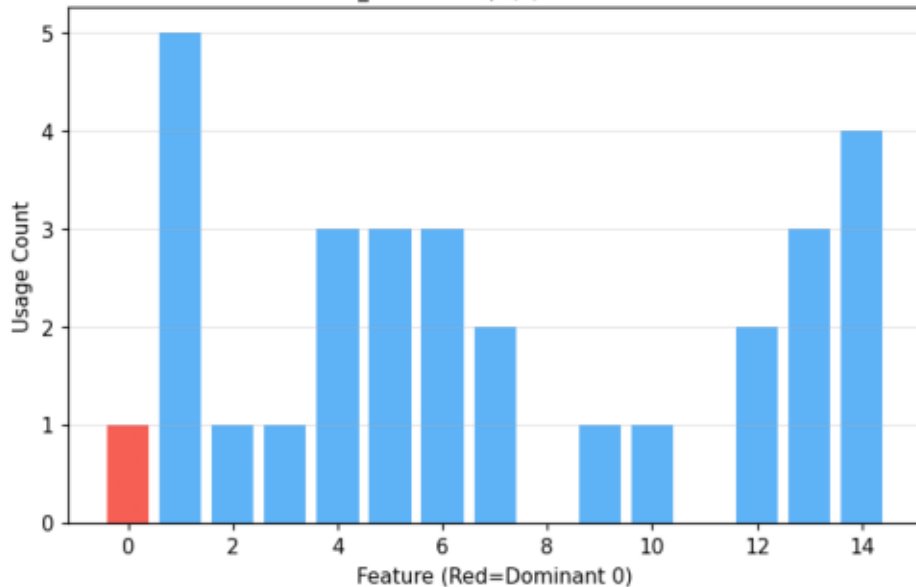
Generalization Performance

- $\text{max_features}=\sqrt{d}$: best generalization (balanced diversity).
- $\text{max_features}=2$: too restrictive, may miss dominant feature too often.
- $\text{max_features}=\text{None}$: degenerates to Bagging (correlated trees).

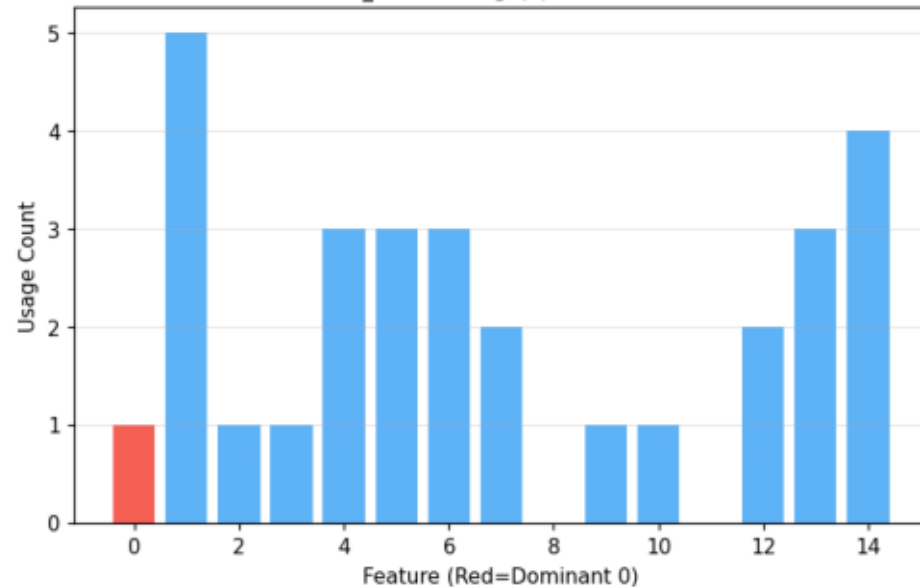
Q2-B: Feature Usage Frequency under Different max_features Settings

Feature Dominance: Feature Usage per max_features Setting

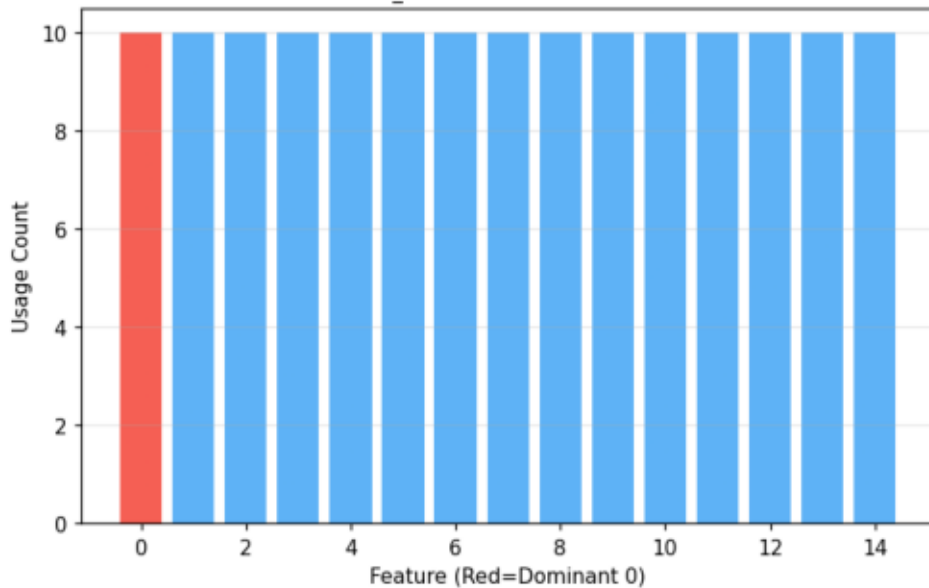
max_features=sqrt(d) acc=0.6056



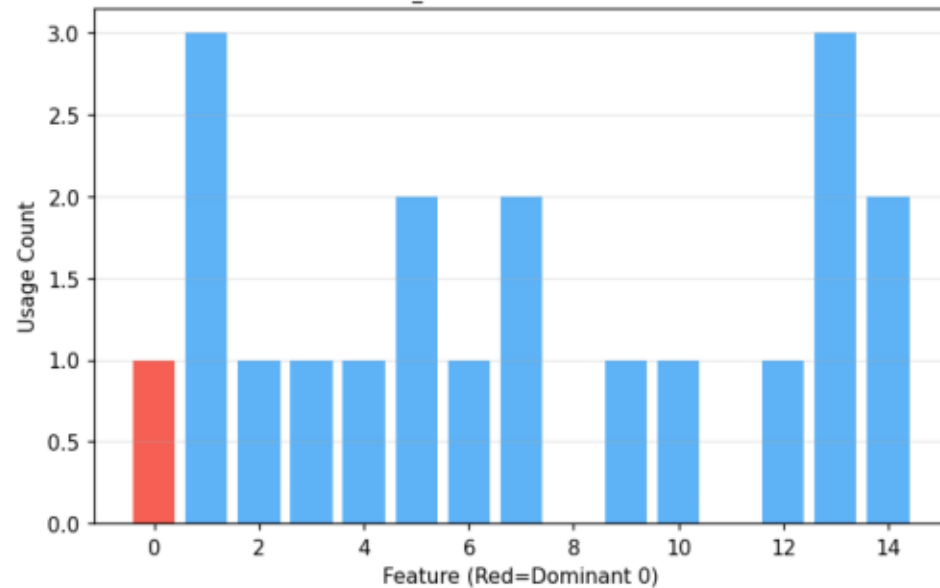
max_features=log2(d) acc=0.6056



max_features=All acc=0.9944

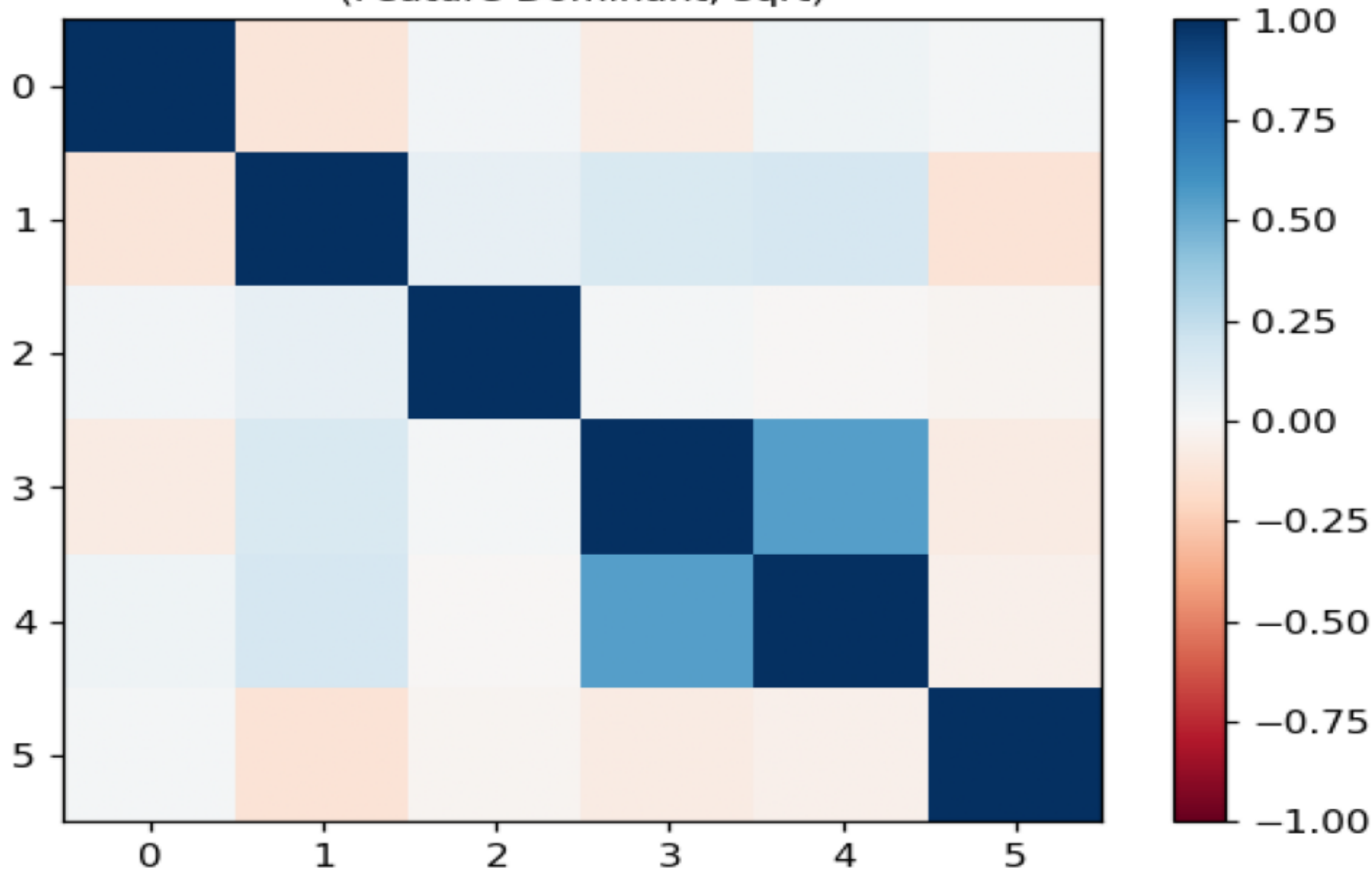


max_features=2 acc=0.6222



Q2-B: Tree Prediction Correlation (Feature Dominant, sqrt subsampling)

Tree Prediction Correlation
(Feature Dominant, sqrt)



How OOB Works

- Each sample appears in $\sim 63.2\%$ of bootstrap samples.
- For a given sample i , $\sim 36.8\%$ of trees never trained on it.
- OOB prediction for sample i = majority vote of those 'unseen' trees.
- OOB accuracy $\approx$ unbiased estimate of generalization error.

Why OOB Works

- OOB trees are 'fresh' — they never saw sample i during training.
- Equivalent to cross-validation but without re-training.
- OOB error converges to true generalization error as $T \rightarrow \infty$.

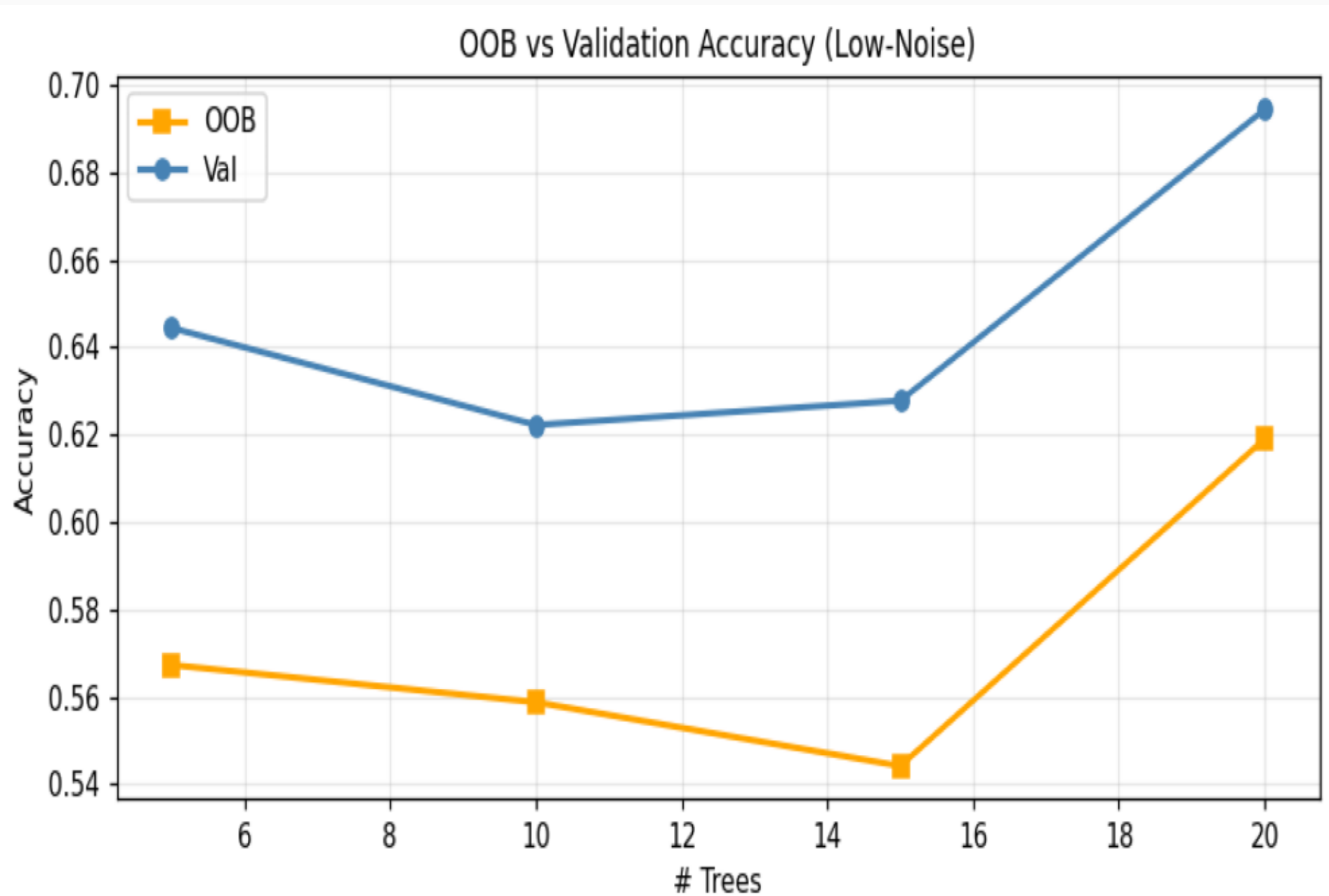
When OOB Becomes Unreliable

- Very small T : few trees are OOB for each sample $\rightarrow$ high variance.
- Small n : some samples may have very few or zero OOB trees.
- Extreme bootstrap with replacement (few unique samples): less OOB diversity.
- Highly imbalanced datasets: OOB sample composition may differ from train.

Observations

- OOB accuracy closely tracks validation accuracy.
- Both stabilize after ~ 30 trees.
- Small gap ($\text{OOB} \leq \text{Val Acc}$) expected because validation set is held-out fully.
- OOB is a reliable free estimator — no need for a separate validation set.

Q2-C: OOB Score vs Validation Score across Number of Trees



Q2-D: High-Dimensional Analysis

Setup

- $n=1000$, $d=105$ (5 informative + 100 noise features).
- Varied number of trees: 5, 10, 20, 30, 50.

Robustness to Noisy Features

- RF with $\text{sqrt}(d) \approx 10$ features per split:
 - Informative features selected $\sim 10/105 \approx 9.5\%$ of the time per split.
 - Despite low probability, RF correctly identifies important features.
- Compared to plain Bagging: RF maintains higher accuracy on noisy data.

Training Time

- Scales roughly $O(T \times n \times d \times \log(n))$ for trees.
- Time increases linearly with T (number of trees).
- High d increases per-split computation, but `max_features` caps this.

Memory Usage

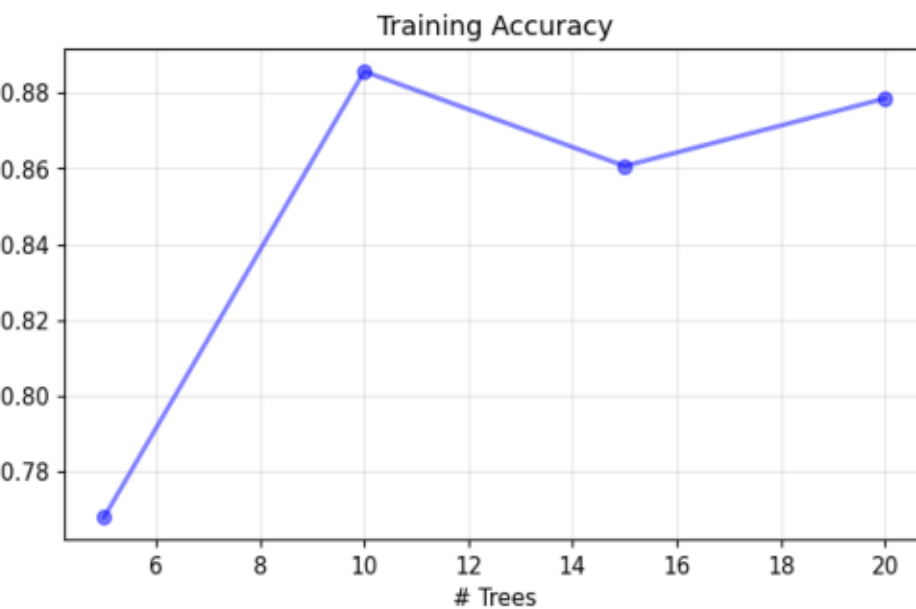
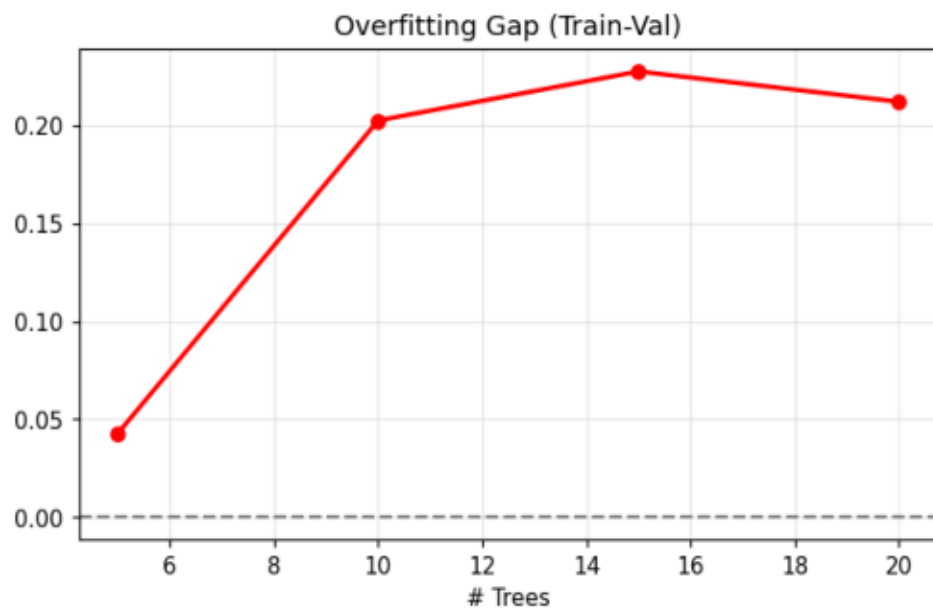
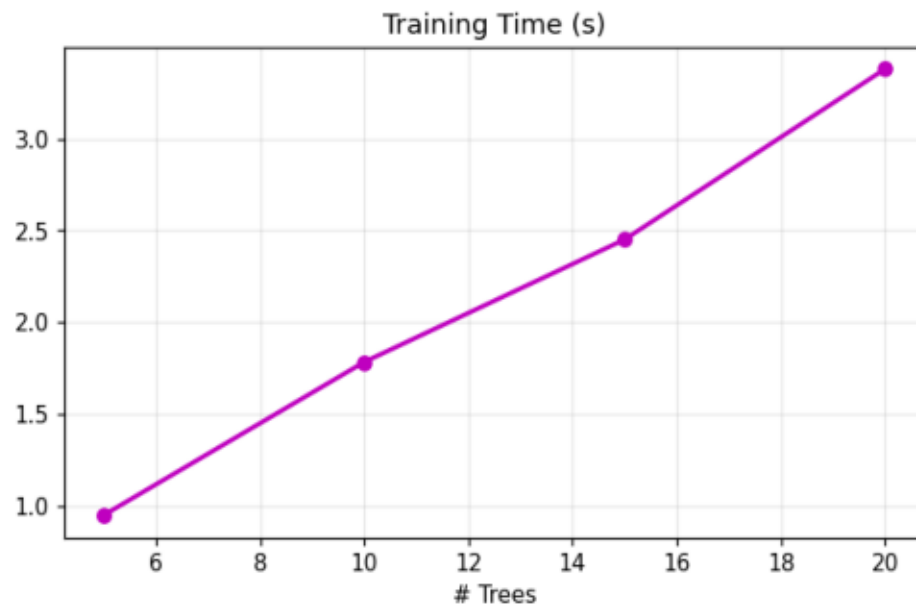
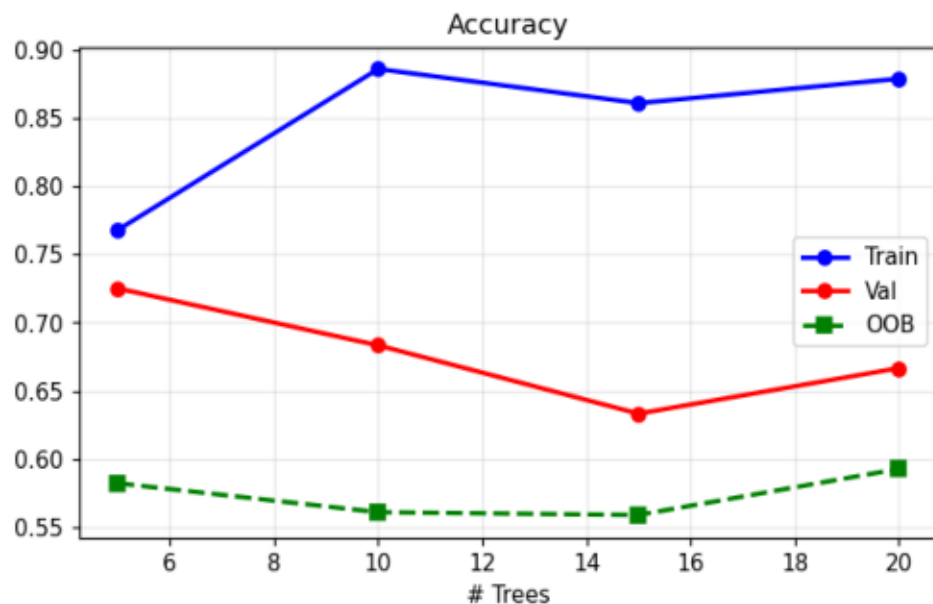
- Memory scales with T and tree depth.
- Storing 1000-feature datasets requires more memory.
- RF is memory-efficient because feature subsampling reduces tree size.

Overfitting Behavior

- Deep trees on high- d data can memorize training set (high train acc).
- RF averaging reduces overfitting gap significantly.
- Adding more trees reduces variance without increasing bias.
- Train-Val gap narrows as T increases.

Q2-D: RF on High-Dimensional Data - Accuracy, Time, Memory, Overfitting

RF High-Dimensional (5 info + 50 noise)



Question 3

Boosting and Sequential Error Correction

Algorithm

1. Initialize: $w_i = 1/n$ for all samples.
2. For $t = 1..T$:
 - Draw weighted sample from distribution w .
 - Train weak learner h_t on weighted sample.
 - Compute weighted error: $\epsilon_t = \sum w_i \cdot 1[h_t(x_i) \neq y_i]$.
 - Compute alpha: $\alpha_t = 0.5 \times \log((1 - \epsilon_t) / \epsilon_t)$.
 - Update weights: $w_i \leftarrow w_i \times \exp(-\alpha_t \times y_i \times h_t(x_i))$.
 - Normalize weights: $w \leftarrow w / \sum w$.
3. Final prediction: $H(x) = \text{sign}(\sum \alpha_t \times h_t(x))$.

Key Properties

- Alpha is positive when $\epsilon_t < 0.5$ (better than random).
- Strong learners get high alpha; weak learners get low alpha.
- Misclassified samples get higher weights → focus on hard cases.
- Error reduces sequentially: bias decreases round by round.

Observations

- On low-noise data: rapid convergence, high accuracy.
- Alpha per round: early rounds have high alpha (easy wins).
- Later rounds: alpha decreases as remaining errors become noise.
- Weighted error stays < 0.5 throughout (all learners better than random).

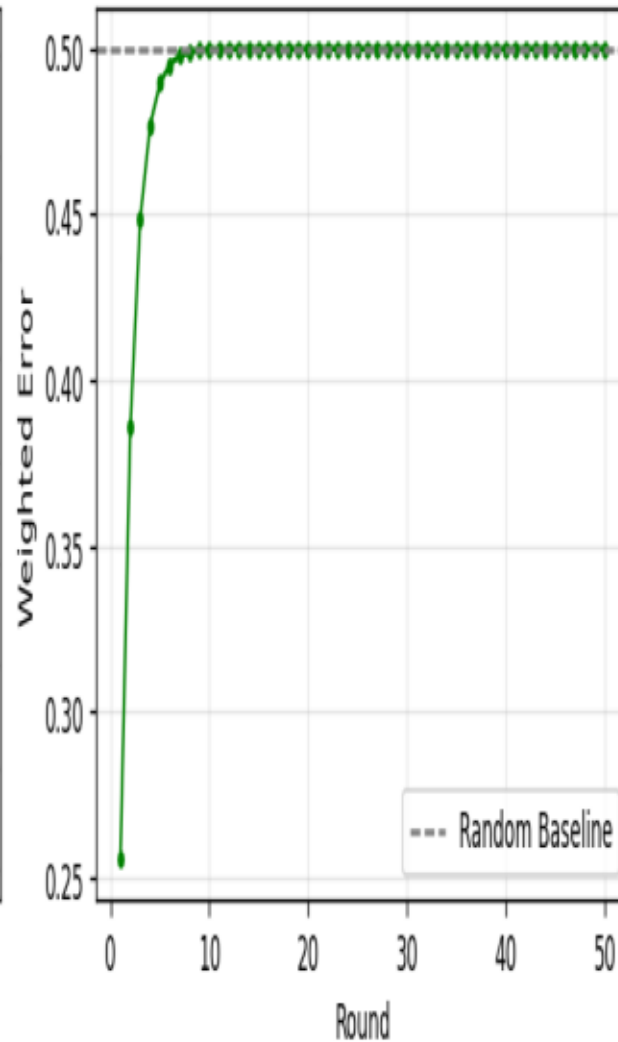
Q3-A: AdaBoost Learning Curve, Errors, and Alphas (Low-Noise Dataset)

AdaBoost (Decision Stumps) - Low Noise Dataset

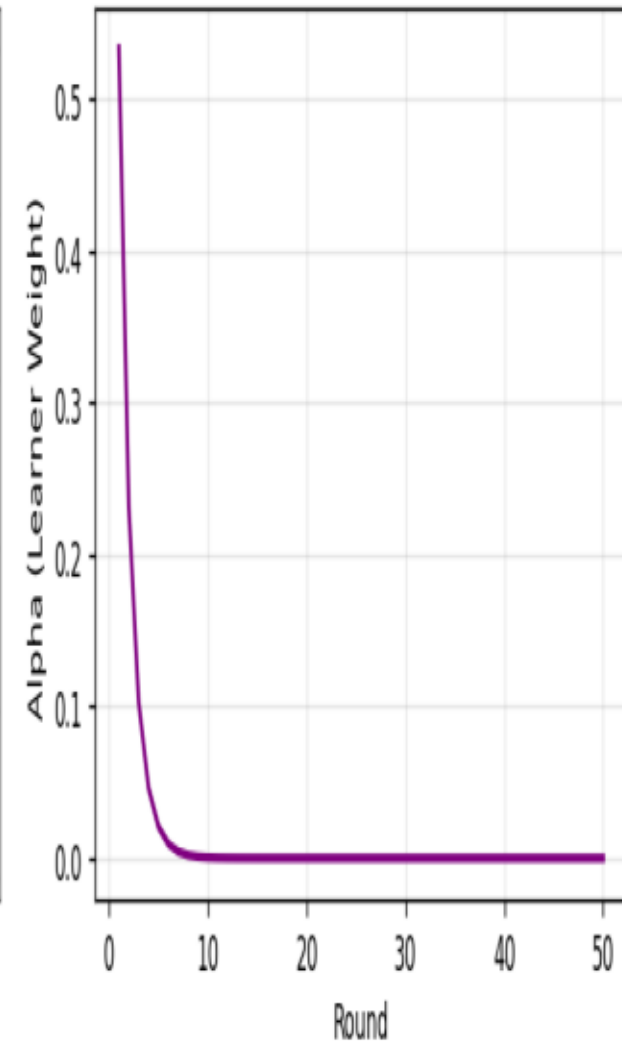
Learning Curve



Weak Learner Error per Round



Alpha per Boosting Round



Label Noise Levels Tested: 5%, 15%, 30%

5% Noise

- Minimal impact on training and validation accuracy.
- AdaBoost converges smoothly and achieves good generalization.
- Weight distribution stays relatively uniform.

15% Noise

- Training accuracy stays high but validation accuracy shows instability.
- Noisy samples begin to receive disproportionately large weights.
- Overfitting starts to appear after many rounds.

30% Noise

- Severe degradation: AdaBoost overfits the noisy labels.
- Sample weights concentrate on mislabeled samples → pathological.
- Training accuracy stays high while validation drops.
- Weight distribution becomes extremely skewed.

Why Boosting Overfocuses on Noisy Samples

- Noisy (mislabeled) samples are inherently 'hard' to classify correctly.
- AdaBoost keeps increasing their weights each round.
- Subsequent learners try hard to classify these impossible samples.
- The model effectively memorizes noise → overfitting.

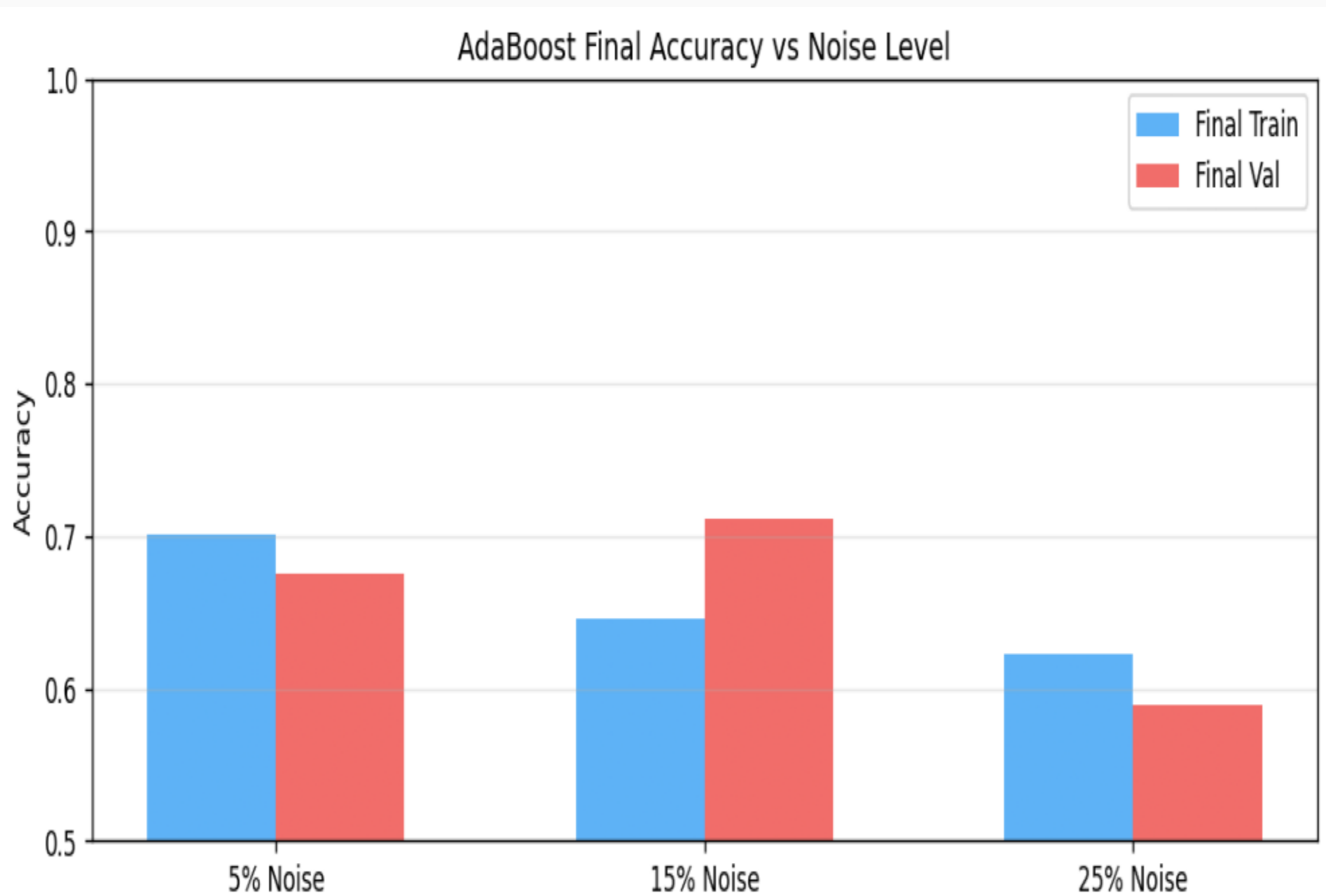
Why Boosting May Overfit Noisy Datasets

- Unlike Bagging, Boosting has no built-in variance reduction.
- Sequential nature means errors compound.

Q3-B: AdaBoost Noise Sensitivity - Learning Curves and Weight Distributions



Q3-B: Summary - Final Accuracy vs Noise Level



Learner Depths Compared: 1 (Stump), 3, 8

Decision Stumps (depth=1)

- High bias individually, but corrected by sequential boosting.
- Smooth learning curves, stable convergence.
- Each stump uses only ONE feature → maximum diversity.
- Less prone to overfitting; AdaBoost adds complexity gradually.

Shallow Trees (depth=3)

- Lower bias than stumps → fewer rounds needed.
- Still benefits from boosting, converges faster.
- Good balance between bias correction and stability.

Deep Trees (depth=8)

- Very low individual bias; each tree almost memorizes training data.
- Alpha values are very high → model changes drastically each round.
- Training accuracy hits 1.0 quickly but validation degrades.
- Boosting with strong learners becomes UNSTABLE.

Why Weak Learners Are Preferred in Boosting

- Weak learners have high individual error → α stays moderate.
- Smooth, stable weight updates prevent weight explosion.
- Diversity maintained across rounds (each stump finds different patterns).

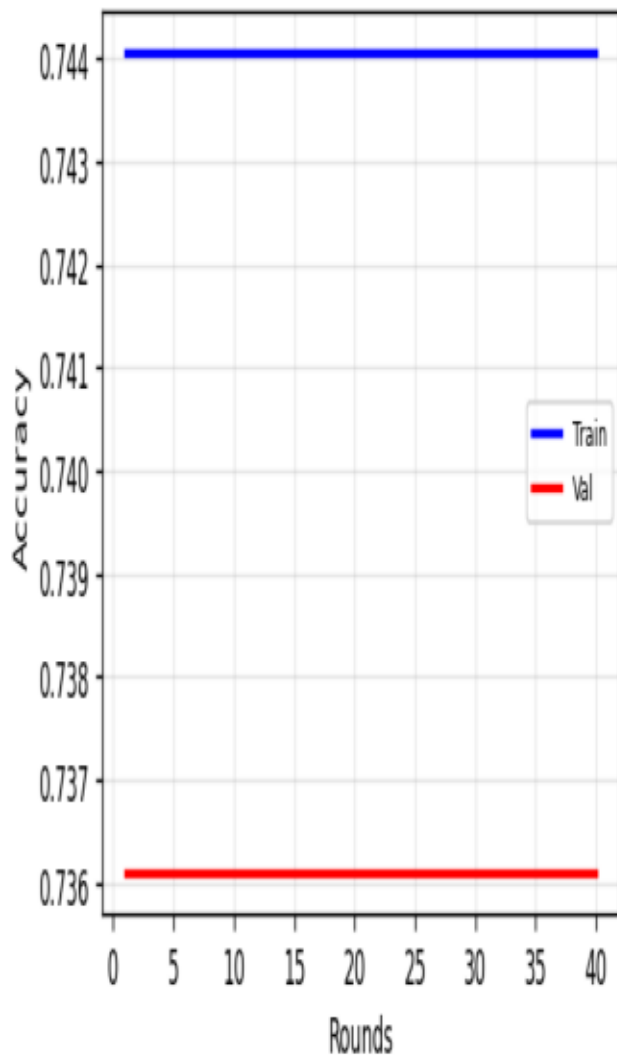
Why Strong Learners Destabilize Boosting

- $\epsilon_t \rightarrow 0$ rapidly → $\alpha_t \rightarrow \infty$ → extreme weight updates.
- A few samples dominate weight distribution after first round.

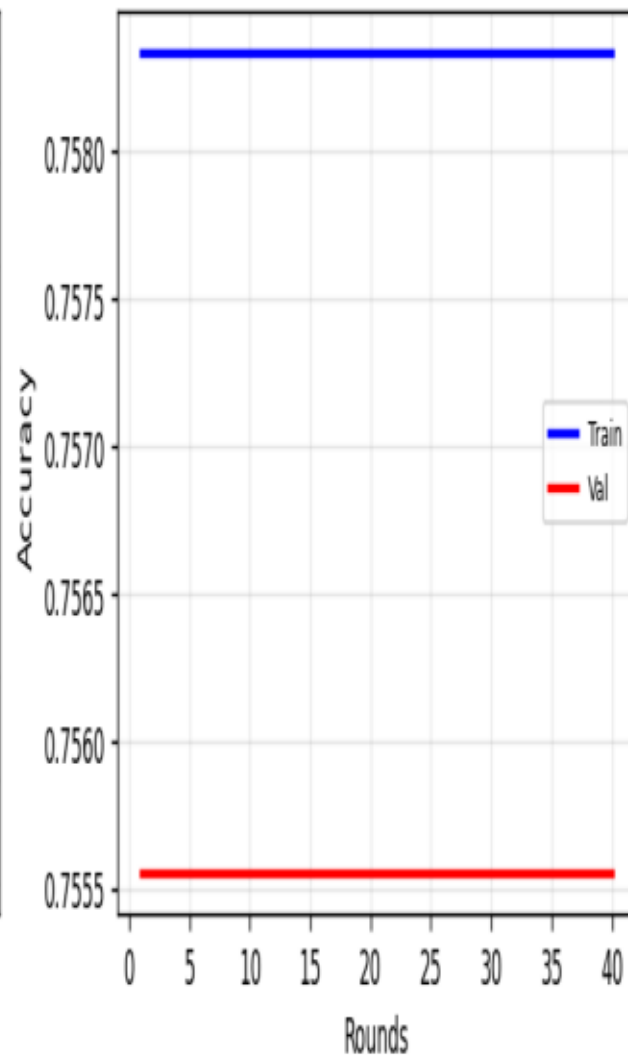
Q3-C: Boosting with Stump vs Shallow vs Deep Base Learners

AdaBoost: Effect of Base Learner Depth

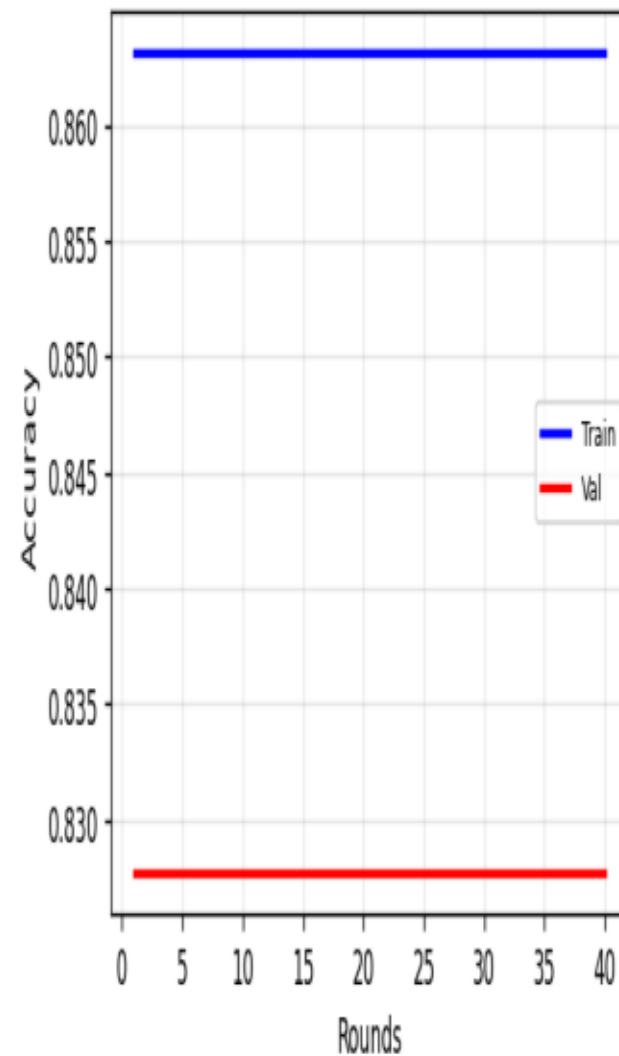
Stump (depth=1)



Shallow (depth=2)



Deep (depth=4)



(a) Why does Boosting reduce bias more than Bagging?

- Bagging averages parallel independent models → bias unchanged.
- Boosting trains SEQUENTIALLY: each model corrects previous errors.
- Focuses on hard examples via weight updates → bias reduced each round.
- Boosting explicitly minimizes training error (exponential loss).

(b) Why are Random Forests more robust than Boosting?

- RF is parallel (independent trees) → errors don't compound.
- Boosting sequential → one bad round propagates to next.
- RF handles noise well (averaging washes out noisy predictions).
- Boosting amplifies noisy samples (assigns them high weights).
- RF has built-in regularization via feature subsampling.

(c) Why can ensembles still overfit despite averaging?

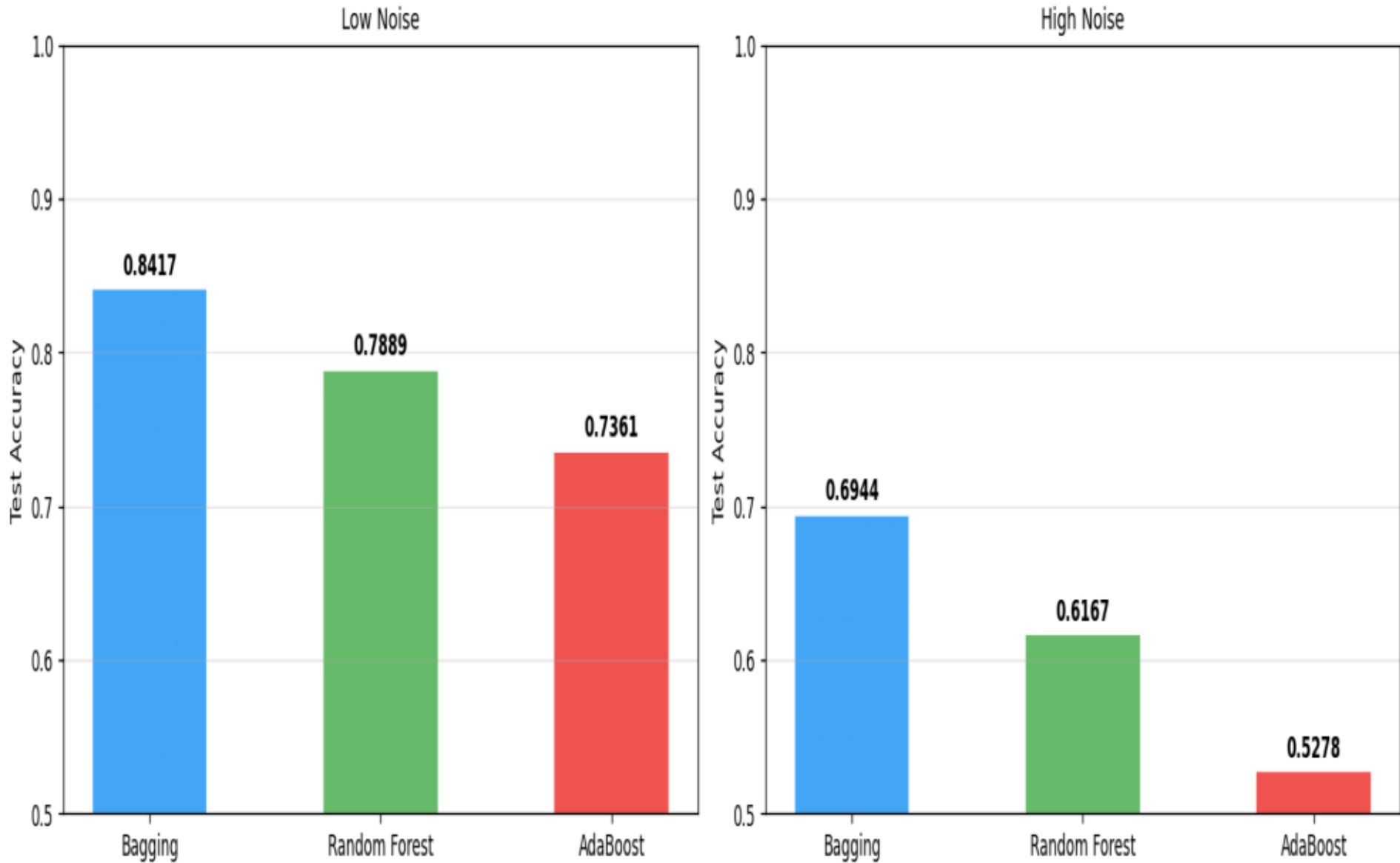
- If base learners all overfit in the SAME WAY, averaging doesn't help.
- With insufficient data, all trees memorize same noise patterns.
- Boosting: noisy labels get concentrated weights → memorization.
- Even averaging 1000 identical overfit models = one overfit model.
- Key: diversity is required for averaging to reduce overfitting.

(d) Why is Boosting sequential while Bagging is parallelizable?

- Bagging: each tree is independent → train all T trees simultaneously.
- Boosting: tree $t+1$ needs error from tree t to update weights.
- Weight update formula: $w_i^{(t+1)} = f(w_i^{(t)}, h_t(x_i))$.
- This SEQUENTIAL DEPENDENCY prevents parallelization.
- Consequence: Boosting scales poorly; Bagging/RF scales linearly with CPUs.

Q3-D: Bagging vs Random Forest vs AdaBoost - Final Accuracy Comparison

Bagging vs Boosting: Accuracy Comparison



Question 1: Bagging

- Bootstrap creates diversity → reduces prediction variance.
- Bagging helps when base learner has high variance (deep trees + noisy data).
- Bagging fails when trees are correlated (low-variance signal, clean data).
- OOB fraction ≈ 0.368 validates bootstrap theory.

Question 2: Random Forests

- Feature subsampling adds extra diversity beyond bootstrap.
- Outperforms Bagging on high-dimensional noisy datasets.
- OOB score is a reliable free proxy for validation accuracy.
- High-dimensional data: RF is robust; training time scales linearly with `n_trees`.
- Feature dominant datasets: sqrt subsampling prevents single-feature domination.

Question 3: AdaBoost

- Sequential error correction effectively reduces bias.
- Noise sensitivity is the primary failure mode.
- Weak learners (stumps) preferred: stable updates, diverse splits.
- Strong learners in boosting → unstable weight dynamics.
- RF more robust to noise; Boosting better on clean low-noise data.

Implementation Notes

- All algorithms: NumPy only (no sklearn ensemble APIs).
- Decision tree built from scratch with Gini/Entropy split criteria.
- OOB computed manually for both Bagging and Random Forest.
- AdaBoost includes staged prediction for learning curve analysis.
- Seed = 4 used throughout for reproducibility.